从 Spec 驱动转向环境与验证驱动——我对 AI Coding 的一点思考

吴佐衍 阿里技术 Jul 31, 2026, 3:18 AM 浙江 60 listened



这是2026年的第 46 篇文章

（ 本文阅读时间：约 15 分钟 ）

最近观察到一个现象：多数同学做 AI Coding 研发提效的第一反应，是把现有的研发流程 workflow 化，让 AI 在每个节点上提效。这条路有现实价值，但我认为它不是 AI Coding 的最终形态。写这篇文章的目的是表达我对 AI Coding 的理解与判断，欢迎拍砖。

01
Coding 正在被解决

过去一年多的时间里，SOTA 模型把 HumanEval、LiveCodeBench、SWE-bench Verified 等 Coding benchmark 相继刷爆，即使是更复杂的 SWE-bench Pro 和 Terminal-Bench 2.1，也已经冲到了 80% 上下。

主流模型 Coding benchmark 成绩（2026 年 7 月快照，各来源口径不一，仅供参考）：

公司	模型	SWE-bench Pro	Terminal-Bench 2.1	Terminal-Bench 2.0	口径
Anthropic	Claude Mythos 5	80.3%	88.0%	—	公开对照成绩；受限模型
OpenAI	GPT-5.6 Sol	64.6%	88.8%	—	官方自报；Ultra 推理档 91.9%

Google	Gemini 3.5 Flash	55.1%	76.2%	—	Google 官方，Terminus-2
Alibaba	Qwen3.7-Max	60.6%*	—	69.7%	官方自报；Pro 使用修正版任务集
Z.ai	GLM-5.2	62.1%	81.0%	—	官方自报；Pro 使用 OpenHands
Moonshot	KIMI 3	—	88.3%	—	NVIDIA 对原生 INT4 的独立测试；NVFP4 为 72.5%
Meta	Muse Spark 1.1	61.5%	80.0%（厂商）/ 69.29±0.99%（独立）	—	Pro 来自 Scale；TB 独立结果来自 Vals
DeepSeek	DeepSeek V4 Pro Max	55.4%	—	67.9%	官方模型卡；Max 为推理档位

不管看官方技术报告还是独立测试，头部厂商的成绩都已经冲上了高分段，**分数差距已经小于口径差距。**Claude Code 的作者 Boris 说"Coding 已经被解决"，单看 benchmark，这句话基本成立。我们内部的数据也在印证同一件事：团队的 AI 生码占比正在快速提升。

02
那为什么效率没有同步上涨

既然模型的 Coding 能力突飞猛进，**我们的研发效率跟上来了吗？至少目前看，答案是没有。**举个我自己的例子：一个 C 端用户动线改造需求终端改码加本地验证，1 个小时就能搞定；但要让它在线上生效，涉及多个内部平台的关联发布，又恰好遇上 618 封网——这个需求从编码完成到真正上线，实际用了 3 周。



按我们团队内部的粗略统计，生码在整个研发链路里只占 20%~30%，甚至可能更少。阿姆达尔定律告诉我们，把一个占三成的环节压缩到接近零，整条链路的提升上限也只有三成。随着

Coding 被解决得越彻底，Coding 之外的部分称为新的瓶颈。

> 阿姆达尔定律（Amdahl's Law）是**计算机科学中用于评估系统优化或并行计算效果的核心法则**。它指出：系统整体的性能提升受限于程序中**无法被优化的串行部分**。即使将某一部分的执行速度提升至无限快，整体性能提升依然存在上限。

另外还有两个常被提到的原因：一是大模型基于公开语料训练，天然缺少我们的业务领域知识；二是 benchmark 分数高，不等于生产环境下可以直接把活干好。这两点都成立，但请注意——它们和上面的案例指向同一个方向：**瓶颈不在 Coding，而在 Coding 之外的其他环节。**

03
诊断：可验证的反馈——AI 的能力边界

为什么偏偏是 coding 被最先解决？表面的答案是它有海量公开代码、公开 issue、公开评测；更深一层的规律是：**AI 总是优先解决那些反馈公开、验证可以规模化的问题。** Coding 和数学最先被突破，不是因为它们简单，而是因为它们是少有的「对错机器自己就能判」的领域：编译是否通过、测试是否通过，跑一下就知道。而企业级生产环境里：答案藏在内部的各个研发系统里，没有公开反馈，验证一次的成本还很高。

这个规律的背后告诉我们：既然 AI 的边界是由「可验证的反馈」确定的，我们要做的是**主动把内部的研发工作流改造成"有反馈、可验证"的环境**。

1992 年，Jack Reeves 在《Code as Design》里提出过一个判断：**源代码才是软件真正的设计，文档只是设计过程中的摘要，天然会丢信息**。这个判断在 AI 时代有了新的读法：生成代码会快到近乎免费，但把模糊的业务意图变成完整、精确、能跑起来的表达，并且验证它真的符合预期。

Coding 正在被解决，恰恰意味着「让 AI 更会写代码」不再是我们的主战场。我们的主战场在验证与环境。

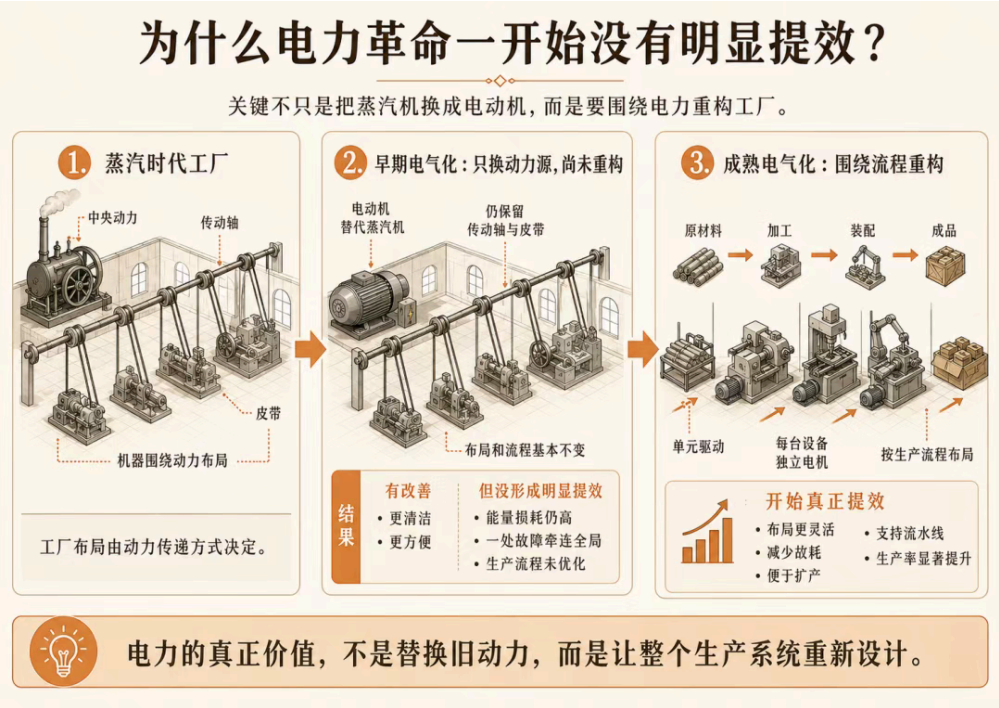
04
历史的快照：电力替换蒸汽机

类比科技革命的历史，在电气化时代，每家工厂的第一步都是拿发电机换下蒸汽机、传动轴布局原样保留。这种方式容易立项、见效快，也让团队积累了跟电力协作的手感。

19世纪末的工厂原本围绕蒸汽机或水轮机设计：一台大型动力机带动贯穿厂房的传动轴，再通过皮带和滑轮连接每台机器。机器必须沿着传动轴排列，整套生产系统的空间布局、作业顺序和管理方式，都由“如何传递动力”决定。

电动机出现后，大多数工厂并没有推倒重建，而只是用一台大型电动机替换蒸汽机，继续带动原有的传动轴和皮带。这种做法被称为「电力轴传动」或「分组驱动」：能源变了，但厂房布局、机器排列和生产流程基本没有改变。旧系统中的能量损耗、整片设备联动、局部故障导致大面积停工等问题仍然存在，电力只是成为了更方便的动力源，还没有成为重构生产系统的基础。

企业之所以不愿立即改造，并不是没有看到电力的潜力，而是受到既有资本的约束。厂房、设备、传动轴和生产线都还能使用，直接废弃意味着巨额资产损失；重新布线、购买小型电机、搬迁机器并调整生产组织，又需要新的投资和停产成本。与此同时，早期电网覆盖有限，电价、供电可靠性、小型电机性能和技术标准也仍在完善。因此，从1899年到1919年，美国制造业电力占机械动力的比例才由约5%上升到50%，扩散本身就经历了大约二十年。

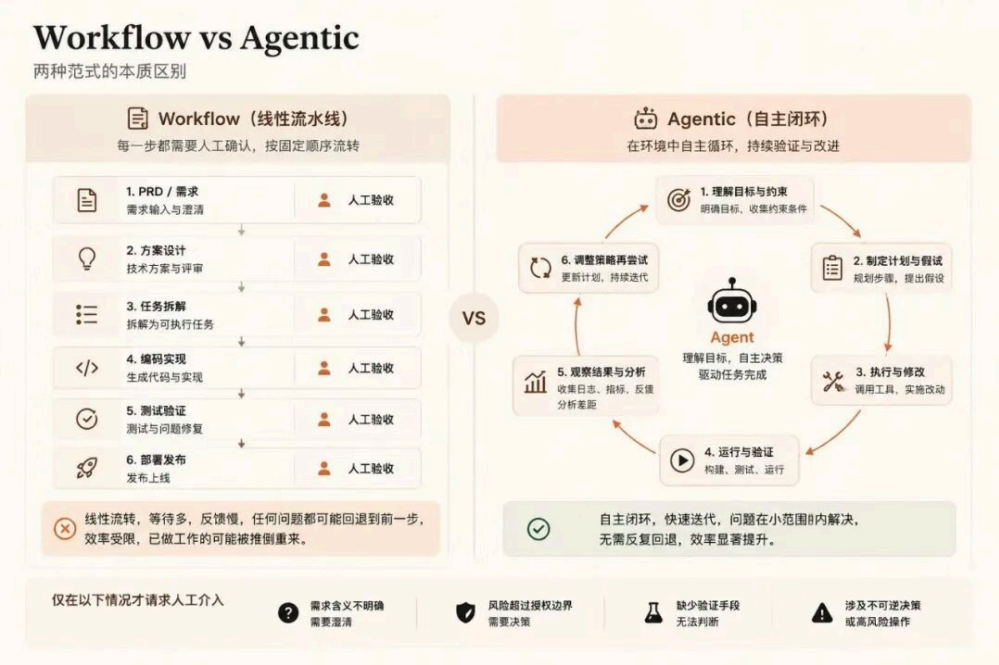


真正的变化发生在企业开始采用「单元驱动」之后：**每台机器配备独立电机，机器不再围绕传动轴排列，而可以按照原材料进入、加工、装配和成品输出的顺序重新布局。**工厂由多层、狭窄、依赖承重传动轴的建筑，转向更加开阔的单层空间；流水线、连续生产、起重设备以及更灵活的材料搬运由此成为可能。电力带来的最大收益，不是蒸汽机与电动机之间的效率差，而是生产流程、厂房结构、设备组织和管理制度的整体重构。美国制造业生产率的明显跃升也主要出现在这些互补改造逐渐普及的20世纪20年代。

05
我们该如何基于 AI 重塑我们的研发工作流

目前的主流做法，是把 AI 嵌进一套人类设计的固定流程：PRD → 需求澄清 → 技术方案 → 任务拆解 → 生码 → 测试，每个节点调用 AI 做局部提效，节点之间的验收和流转还是按老流程走。SDD、各种 Agent Skills、Superpowers 这类插件，本质上都是同一条路线。

这条路线有一个结构性天花板：**它优化的是「人使用 AI 的方式」，而不是「AI 使用环境的方式」。**每个节点都等人验收、按人画的管道流转，AI 实际上被困在条条框框里。因为这种方式是给人协作分工的产物，固化成 Agent 必须遵守的执行顺序，本质上是在 AI 时代重新引入瀑布模型，它假设需求澄清后就是对的、方案评审后就是可行的、实现阶段只需忠实执行。而真实的开发里，一次测试失败可能来自实现、环境、数据、架构假设或需求理解中的任何一层；任何失败都回退到方案重新生码，只会把已经做对的判断一起推倒。而且，**按照 Jack Reeves 代码即设计的逻辑，技术方案本身并不等价于代码。**



另一条路线是 agentic：Agentic 的本质不是「AI 主导决策」，而是「AI 能否自主获取验证信号」，不用每一步都等人来判断对不对。在这条路线里，workflow 并不消失，它退到自己该待的位置：调度、权限控制、状态保存、审计和高风险检查。电气化真正的跃升，不是换发电机，是单元驱动：让每个工位摆脱中央传动轴，获得独立运转的能力。我们要做的，是让 Coding 这个工位先独立转起来。

06
如何获取模型能力溢出的红利——我们在 Bet 什么

两条路线怎么选，落到日常就是一笔投资账。我可以用一个问题，考虑所有 AI 相关的投入：

「下一代 SOTA 模型发布时，我正在做的这件事，是获益，还是作废？」

按这个标准，投入可以分成两类：

- **贬值区**：围绕「生成能力」的投入：微调模型提升生码质量、囤积提示词技巧、精细编排生码流程。过去一年反复上演的剧情是：昨天的脚手架，变成今天模型的内置能力。在这条线上投入，等于和模型厂商的军备竞赛对赌。
- **增值区**：围绕「环境与验证」的投入：把企业内部的构建、部署、测试、数据、接口、日志、监控、发布链路，做成 AI 可调用的工具。这件事没有任何厂商能替我们做，而且模型越强，这些资产越值钱：同样的环境，更强的模型能跑出更深、更长的自主循环。

贬值和增值的分界线，不在「编排」还是「环境」这两个词上，在于它和模型能力的关系上：

- **凡是替代或补偿模型推理能力的东西——拆解步骤、设计提示词、编排流程——都会被模型的下一次升级吸收，因为模型厂商恰恰是在这些维度上训练下一代模型；**
- **凡是给模型提供它自己造不出来的信息的东西——内部系统的真实状态、构建结果、日志、测试反馈都会持续增值，因为关于世界的信息只能从世界里来，不可能从权重里长出来。**



为什么环境会持续增值，而生成投入不会？还可以用一个简单的关系来理解：**企业从 AI 拿到的红利，约等于模型能力与环境能力的乘积，而不是和。** 模型是这个乘积里租来的因子，跟着厂商的节奏自己往上涨；环境是自有的因子，只能靠我们自己建，而且只涨不跌。乘法的意思有两层：任何一端是零，结果就是零——再强的模型，进不了我们的环境，生产力就是零；反过来，模型每变强一代，都会把同一套环境的价值重新放大一遍。workflow 化的收益是线性的，因为它只是给现有环节加了个常数；环境投入的收益是复利的，因为它在乘法里占住了一个因子。

所以我的主张是：不做「如何让 AI 生码更强」的军备竞赛，只做企业内部环境的工具化。 这不是说模型相关工作一概不碰，模型选型、评测、上下文接入当然要做，划界标准就是上面那个问题。

07
环境与工具才是 Agent 的核心基础设施

- **业务领域知识的构建**：业务域 SKILL，本体知识库构建等。
- **可复现、可调用的研发环境：** 让 Agent 能独立构建项目、启动服务、准备数据、调用接口、操作浏览器或 APP、查看日志和调用链、读取监控指标。本质是把「只有人会操作的内部系统」，翻译成「AI 可调用的工具」。
- **分层验证体系：**秒级反馈（编译、类型检查、单元测试）适合高频自主迭代；分钟级反馈（集成测试、契约测试、浏览器自动化）覆盖更真实的行为；人工判断只保留给机器无法可靠裁决的部分：业务价值、用户体验、伦理边界。
- **端到端的研发系统打通**：我们的研发流程涉及一系列系统，需要将这些系统改造为 AI Friendly 的模式，例如，发布平台、实验平台、监控平台等
- **spec 与技术方案的新写法：区分「约束」与「假设」。** 数据不能出域、接口必须向后兼容、延迟不能超过阈值，这些是约束，要长期保存并尽可能自动检查；微服务还是单体、用哪

种缓存策略，这些是有待验证的假设，应允许 AI 根据实现和运行中的反馈自己调整。spec 负责划定「什么算对」的边界，不负责规定实现路径。

最后，回头解剖我们自己案例里的这只「麻雀」。 如果编码只占 1 小时，那么跨平台影响面分析、联调环境、发布编排、灰度验证、封网期合规检查的工具化，价值远大于继续挤压编码环节。



**注：本文为作者个人技术思考与经验分享，不代表公司的官方立场或观点。文中部分论述涉及对技术趋势和发展方向的前瞻性判断，基于作者写作时的认知与经验，所有内容仅供交流参考，读者应结合自身场景独立评估。*



欢迎留言一起参与讨论~

内容含AI生成图片

阅读 5701

留言 8

写留言

wzp 北京 7月31日 2

环境优先,事实优先,掌握了fact和环境, 再去理解prd和后续执行

桐 北京 8月2日 1

很不错啊，与“Anthropic在Managed Agent中讲Harness的理念与定位”有异曲同工之妙。

sdb 福建 7月31日

写的真好，希望能多发一些关于aicoding的文章~

阿里技术 作者 8月2日 1

安排！

水听风吟 北京 9月10日

爱看、多发

四点共圆 北京 8月16日

如何做到环境验证呢？沙箱还是测试环境的权限开放给ai

TOM夜 上海 7月31日

非常有启发👍

yonoel 江苏 7月31日

核心还是先做基础设施建设，才能大幅提高生产力。但在解决生产力问题上，先考虑怎么解决方向问题